

# EE-222 Data Structures and Algorithms

BS(CE)-2k21

## LAB REPORT # 6



### Submitted By:

| Reg.<br>No. | Name            |
|-------------|-----------------|
| 21-CE-009   | Huzaifa Shahbaz |

### Department of Computer Engineering

### HITEC University Taxila

|          | Presentation<br>(Format(0.5)+<br>Title (0.5)+<br>Conclusion(1)<br>+Procedure(2)<br>+Objective(1)) | Calculation/<br>Coding | Observation/<br>Program Results | Total |
|----------|---------------------------------------------------------------------------------------------------|------------------------|---------------------------------|-------|
| Total    | 5                                                                                                 | 5                      | 5                               | 15    |
| Obtained |                                                                                                   |                        |                                 |       |

---

Lab Instructor.  
Kaynat Rana

## Experiment #6

### Implementation of Doubly Linked List

#### Objective:

The objective of this session is to understand the various operations on Simple and Circular Doubly Linked list in C++.

1. Insertion
2. Deletion
3. Traversal

#### Equipment Used:

Visual studio C++

#### Procedure:

##### Linked List:

A linked list is a collection of components, called nodes. Every node (except the last node) contains the address of the next node. Thus, every node in a linked list has two components: one to store the relevant information (that is, data) and one to store the address, called the next, of the next node in the list. The address of the first node in the list is stored in a separate location, called the head or first. Figure 1 is a pictorial representation of a node. Each node in the linked list contains:

1. One or more members that represent data (e.g., inventory records, customer names, addresses, telephone numbers, etc).
2. 2 pointers, that can point to previous and next nodes.



Figure 1: Linked List Node

##### Doubly Linked list:

A list of items, called nodes, in which the order of the nodes is determined by the addresses, called the prev and next, stored in each node.

The list in Figure2 is an example of a linked list.

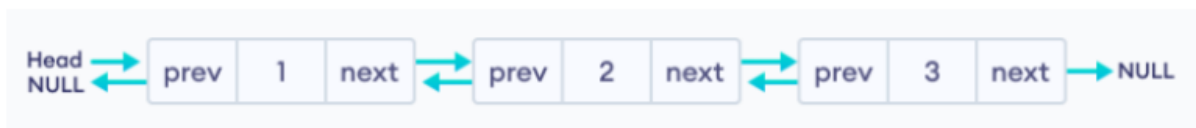


Figure 2: Linked List

The arrow in each node indicates that the address of the node to which it is pointing is stored in

that node. The down arrow in the last node indicates that this next field is NULL. Because each node of a linked list has two components, we need to declare each node as a class or struct. The data type of each node depends on the specific application — that is, what kind of data is being processed. However, the next component of each node is a pointer. The data type of this pointer variable is the node type itself. For the previous linked list, the definition of the node is as follows. (Suppose that the data type is int.)

struct node

```
{  
int data;  
node* prev, *next;  
};
```

The pointer variable declaration is as follows:

```
node* head;
```

### **Operations on Linked List:**

Following operations can be performed on a linked list:-

#### **1. Insertion:**

A node can be inserted at

- i. The start of the list
- ii. End of the list (Appending a node)
- iii. Middle of the list

Consider you have to create a sorted linked list.

#### **i. Appending a Node to the Linked List:**

To append a node to a linked list, means adding it to the end of the list. The append Node member function accepts a float argument, num. The function will -

- a) Create a new node.
- b) Store data in the new node.
- c) If there are no nodes in the list  
Make the new node the first node.

Else

Traverse the List to Find the last node.

Add the new node to the end of the list.

End If.

#### **ii. Inserting Node in the Start and Middle of Linked List:**

Inserting a node in the middle of a list is more complicated than appending a node. Assume all values in the list are sorted, and you want all new values to be inserted in their proper position (preserving the order of the list). We use the same List Node structure again, with pseudo code. This pseudo code shows the algorithm to find the new node's proper position in the list, and inserting it there. It is assumed the nodes already in the list are ordered.

1. Create a new node.
2. Store the data in new node.
3. If there are no nodes in the list  
Make the new node the first node.

1. Else

Find the first node whose value is greater than or equal the new value, or the end of the list (whichever is first).

1. Insert the new node before the found node, or at the end of the list if no node was found.

End If.

## **2. Deletion of a Node:**

This requires 2 steps :

- a) Remove the node from the list without breaking the links created by the next pointers.
- b) Delete the node from memory.

The delete Node member function searches for a node with a particular value and deletes it from the list. It uses an algorithm similar to the insert Node function.

1. The two node pointers nodePtr and previousPtr are used to traverse the list (as before).
2. When nodePtr points to the node to be deleted, previous Node->next is made to point to nodePtr->next.
3. This removes the node pointed to by nodePtr from the list.
4. The final step is to free the memory used by the node using the delete operator.

## **3. Traversing a Linked List:**

The display List is a member function, that traverses the list, displaying the value member of each node. The following pseudo code represents the algorithm -

1. Assign list head to node pointer
2. While node pointer is not NULL

Display the value member of the node pointed to by node pointer.

Assign node pointer to its own next member.

3. End While.

## **Doubly circular linked list:**

A doubly circular linked list is a data structure in which each node contains a pointer to the next and previous node in the list, and the last node points back to the first node, creating a circular structure. In a doubly circular linked list, each node has two pointers, one to the next node and one to the previous node.

## **Tasks**

### **Task 1:**

Write a C++ program to create Doubly Linked List ADT. Your Linked List must be able to perform the following functionalities:

- a) Insertion of an integer (start/middle/end) in sorted linked list.
- b) Deletion of a given integer from the above linked list.
- c) Display the contents of the above list after deletion.

## CODE:

```
1 // DUBLY LINKED LIST
2 using namespace std;
3 struct Node
4 {
5     int data;
6     Node* prev;
7     Node* next;
8 };
9 class DoublyLinkedList
10 {
11 private:
12     Node* head; // Pointer to the head of the list
13 public:
14     DoublyLinkedList()
15     { // Constructor to initialize an empty list
16         head = NULL;
17     }
18     void displayList()
19     { // Function to display the contents of the list
20         Node* current = head;
21         while (current != NULL)
22         {
23             cout << current->data << " ";
24             current = current->next;
25         }
26         cout << endl;
27     }
28     void insert(int value)
29     { // Function to insert an integer in sorted order
30         Node* newNode = new Node;
31         newNode->data = value;
32         newNode->prev = NULL;
33         newNode->next = NULL;
34         if (head == NULL)
35         { // Case 1: List is empty
36             head = newNode;
37         }
38         else if (value <= head->data)
39         { // Case 2: Insert at the beginning
40             newNode->next = head;
41             head->prev = newNode;
42             head = newNode;
43         }
44         else { // Case 3: Insert in middle or end
45             Node* current = head;
46             while (current->next != NULL && current->next->data < value) {
47                 current = current->next;
48             }
49             newNode->next = current->next;
50             if (current->next != NULL)
51             {
52                 current->next->prev = newNode;
53             }
54             current->next = newNode;
55             newNode->prev = current;
56         }
57     }
58     void remove(int value)
59     { // Function to delete a given integer from the list
60         Node* current = head;
61         // Case 1: Value to be removed is at the head of the list
62         if (head != NULL && head->data == value)
63         {
64             head = head->next;
65             if (head != NULL)
66             {
67                 head->prev = NULL;
68             }
69             delete current;
70             return;
71         }
72         // Case 2: Value to be removed is in the middle or end of the list
73         while (current != NULL && current->data != value)
74         {
75             current = current->next;
76         }
77         if (current == NULL)
78         { // Value not found in list
79             return;
80         }
81         current->prev->next = current->next;
82         if (current->next != NULL)
83         {
84             current->next->prev = current->prev;
85         }
86         delete current;
87     }
88 };
89
90 int main()
91 {
92     DoublyLinkedList myList;
93     myList.insert(5);
94     myList.insert(10);
95     myList.insert(2);
96     myList.insert(8);
97     myList.insert(3);
98     cout << "List after insertion: ";
99     myList.displayList();
100     myList.remove(8);
101     cout << "List after deletion: ";
102     myList.displayList();
103     return 0;
104 }
```

## OUTPUT:

```
Microsoft Visual Studio Debug Console

list after insertion: 1 2 5 8 10
list after deletion: 1 2 5 10

E:\dev c++\DOUBLYLINKEDLIST\x64\Debug\DOUBLYLINKEDLIST.exe (process 20424) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

## Task 2:

Write a C++ program to create Circular Doubly Linked List ADT. Your Linked List must be able to perform the following functionalities:

- Insertion of an integer (start/middle/end) in sorted linked list.
- Deletion of a given integer from the above linked list.
- Display the contents of the above list after deletion.

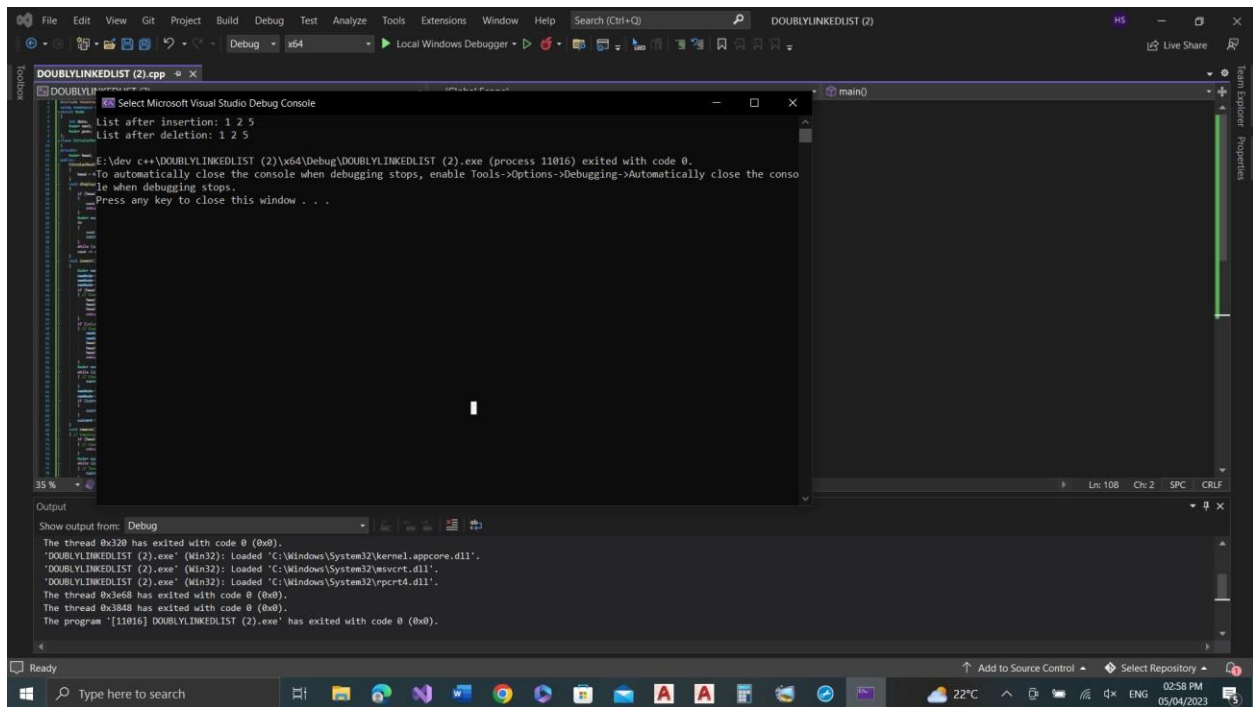
## CODE:

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node* next;
    Node* prev;
};

class CircularDoublyLinkedList
{
public:
    Node* head;
    CircularDoublyLinkedList()
    {
        head = NULL;
    }
    void displayList()
    {
        if (head == NULL)
        {
            cout << "List is empty" << endl;
            return;
        }
        Node* current = head;
        do
        {
            cout << current->data << " ";
            current = current->next;
        } while (current != head);
        cout << endl;
    }
    void insert(int value)
    {
        Node* newNode = new Node;
        newNode->data = value;
        newNode->next = NULL;
        newNode->prev = NULL;
        if (head == NULL)
        {
            // Case 1: List is empty
            head = newNode;
            head->next = head;
            head->prev = head;
            return;
        }
        if (value == head->data)
        {
            // Case 2: Insert at the beginning
            newNode->next = head;
            newNode->prev = head->prev;
            head->prev->next = newNode;
            head->prev = newNode;
            head = newNode;
            return;
        }
        Node* current = head;
        while (current->next != head || current->next->data < value)
        {
            // Case 3: Insert in middle or end
            current = current->next;
        }
        newNode->next = current->next;
        newNode->prev = current;
        if (current->next == head)
        {
            current->next->prev = newNode;
        }
        current->next = newNode;
    }
    void remove(int value)
    {
        // Function to delete a given integer from the list
        if (head == NULL)
        {
            // Case 1: List is empty
            return;
        }
        Node* current = head;
        while (current->data != value || current->next == head)
        {
            // Traverse the list to find the node to be deleted
            current = current->next;
        }
        if (current->data == value)
        {
            // Case 2: Value not found in list
            return;
        }
        if (current == head)
        {
            // Case 3: Deleting the head of the list
            head = current->next;
            current->prev->next = current->next;
            current->next->prev = current->prev;
            delete current;
        }
    }
};

int main()
{
    CircularDoublyLinkedList myList;
    myList.insert(5);
    myList.insert(10);
    myList.insert(2);
    myList.insert(8);
    myList.insert(3);
    cout << "List after insertion: ";
    myList.displayList();
    myList.remove(8);
    cout << "List after deletion: ";
    myList.displayList();
    return 0;
}
```

## OUTPUT:



```
DOUBLYLINKEDLIST (2).cpp
Select Microsoft Visual Studio Debug Console
List after insertion: 1 2 5
List after deletion: 1 2 5
E:\dev c++\DOUBLYLINKEDLIST (2)\x64\Debug\DOUBLYLINKEDLIST (2).exe (process 11016) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .

Output
Show output from: Debug
The thread 0x120 has exited with code 0 (0x0).
'DOUBLYLINKEDLIST (2).exe' (Win32): Loaded 'C:\Windows\System32\kernel.appcore.dll'.
'DOUBLYLINKEDLIST (2).exe' (Win32): Loaded 'C:\Windows\System32\user32.dll'.
'DOUBLYLINKEDLIST (2).exe' (Win32): Loaded 'C:\Windows\System32\RPCRT4.dll'.
The thread 0x3e68 has exited with code 0 (0x0).
The thread 0x3848 has exited with code 0 (0x0).
The program '[11016] DOUBLYLINKEDLIST (2).exe' has exited with code 0 (0x0).
```

## Conclusion:

In conclusion, a **doubly linked list** is a powerful data structure that can be used in a variety of applications. It provides flexibility and ease of use, making it a popular choice for many programmers. **Doubly circular linked lists** are similar to doubly linked lists, but the last node points back to the first node, creating a circular structure. This allows for traversal in both directions as well as easy access to the first and last nodes.

